

Task

Test task #2 - Data Quality focus

Use the E-Commerce Data dataset:

<https://www.kaggle.com/datasets/carrie1/ecommerce-data>

This dataset contains online retail transactions, including invoices, quantities, prices, customers, and countries.

Your goal is to design five data quality checks that would help ensure this dataset can be reliably used for analytics and reporting.

For each check, specify:

What it verifies

The data quality dimension (e.g., completeness, validity, uniqueness, , accuracy)

The SQL logic or condition

The severity level (warning or critical)

Task Solving

For this task I used PostgreSQL database on IntelliJ Idea.

Firstly, we create the table with 8 columns that the E-Commerce Data dataset has :

```
[2025-10-28 20:50:23] Connected
postgres.public> CREATE TABLE ecommerce (
                                InvoiceNo    TEXT,
                                StockCode   TEXT,
                                Description  TEXT,
                                Quantity    INT,
                                InvoiceDate  TIMESTAMP,
                                UnitPrice    NUMERIC(12,2),
                                CustomerID   TEXT,
                                Country      TEXT
                                )
[2025-10-28 20:50:23] completed in 51 ms
```

Then, import data from .csv file to our ecommerce table :



ecommerce

data.csv imported to **ecommerce**: 541,909 rows (14 sec, 576 ms, 3.58 MB/s)

Let's change the format from timestamp to MM/DD/YYYY

```
postgres.public> ALTER TABLE ecommerce
                  ALTER COLUMN InvoiceDate TYPE TIMESTAMP
                  USING to_timestamp(InvoiceDate, 'MM/DD/YYYY HH24:MI')
[2025-10-28 21:13:03] completed in 2 s 409 ms
```

This is what we got for now :

description ▾	quantity ▾	invoicedate ▾	unitprice ▾	customerid ▾	country
WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00.000000	2.55	17850	United Kin
WHITE METAL LANTERN	6	2010-12-01 08:26:00.000000	3.39	17850	United Kin
CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00.000000	2.75	17850	United Kin
KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00.000000	3.39	17850	United Kin
RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00.000000	3.39	17850	United Kin
SET 7 BABUSHKA NESTING BOXES	2	2010-12-01 08:26:00.000000	7.65	17850	United Kin
GLASS STAR FROSTED T-LIGHT HOLDER	6	2010-12-01 08:26:00.000000	4.25	17850	United Kin
HAND WARMER UNION JACK	6	2010-12-01 08:28:00.000000	1.85	17850	United Kin
HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00.000000	1.85	17850	United Kin

Now, let's dive into making data quality checks that would help ensure this dataset can be reliably used for analytics and reporting.

Quality checks

1. Completeness check

verifies if essential analytical fields are not null or empty

Critical severity level - without it, it is impossible to build customer analytics. If fields like description were NULL then it would be just a warning.

```
SELECT COUNT(*)
FROM ecommerce

WHERE CustomerID IS NULL OR CustomerID = ''
   OR InvoiceNo IS NULL OR InvoiceNo = ''
   OR StockCode IS NULL OR StockCode = ''
   OR Quantity IS NULL
   OR UnitPrice IS NULL
   OR InvoiceDate IS NULL;
```

count ▾

135080

The screenshot shows a database IDE with a SQL query editor on the left and a results pane on the right. The query is a SELECT statement filtering for records where CustomerID, InvoiceNo, StockCode, Quantity, UnitPrice, or InvoiceDate is NULL. The results pane shows the first four rows of the query results.

```

1 SELECT *
2
3 FROM ecommerce
4
5 WHERE CustomerID IS NULL OR CustomerID = ''
6      OR InvoiceNo IS NULL OR InvoiceNo = ''
7      OR StockCode IS NULL OR StockCode = ''
8      OR Quantity IS NULL
9      OR UnitPrice IS NULL
10     OR InvoiceDate IS NULL;

```

The results pane shows the following data:

	invoiceno	stockcode	description	quantity	unitprice
1	541219	21032	SPACE CADET WHITE	1	20.00
2	541811	20713	JUMBO BAG OWLS	1	20.00
3	537220	22450	SILK PURSE BABUSHKA BLUE	1	20.00
4	537230	21733	RED HANGING HEART T-LIGHT HOLDER	1	20.00

To see the rows, use `SELECT *`

2. Validity check

Checks the correctness of the quantity and price :

For regular invoices (invoice number contains only numbers) — quantity > 0 and price per unit > 0;

For cancellations (invoice number starts with C) — quantity < 0, but price per unit > 0.

Shows number of discrepancies in each case

Critical severity level - if here is an error here, it breaks the basic business logic

```

SELECT COUNT(*)
FROM ecommerce
WHERE InvoiceNo NOT LIKE 'C%'
AND (Quantity <= 0 OR UnitPrice <= 0)

UNION ALL

SELECT COUNT(*)
FROM ecommerce
WHERE InvoiceNo LIKE 'C%'
AND (Quantity >= 0 OR UnitPrice <= 0);

```

	count
1	2521
2	0

3. Uniqueness check

shows full rows with duplicates of invoice numbers

Critical severity level - if we count how many invoices we got and use DISTINCT, it will show smaller number. But if it said "invoice number can be repeated, but the lines inside must be different" then it would be warning, and the business key is something like invoice number + stock code.

```
SELECT *
FROM ecommerce
WHERE InvoiceNo IN (
    SELECT InvoiceNo
    FROM ecommerce
    WHERE InvoiceNo IS NOT NULL
    GROUP BY InvoiceNo
    HAVING COUNT(*) > 1
)
ORDER BY InvoiceNo;
```

	invoiceno	stockcode	description	quantity	
1	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	20:
2	536365	71053	WHITE METAL LANTERN	6	20:
3	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	20:
4	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	20:
5	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	20:
6	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	20:
7	536365	85107A	WHITE HANGING HEART T-LIGHT HOLDER	6	20:

4. Timeliness check

Checks whether the data is up to date compared to the current date.

Critical severity level - it is important that data is relevant, fresh and available exactly when it is needed for decision making - for example, in production reporting.

```
SELECT
    MAX(invoicedate) AS latest_transaction_date,
    EXTRACT(DAY FROM (CURRENT_DATE - MAX(invoicedate))) AS
days_since_last_record
FROM ecommerce;
```

The screenshot shows a PostgreSQL IDE interface. The SQL editor contains the following query:

```
1 SELECT
2     MAX(invoicedate) AS latest_transaction_date,
3     EXTRACT(DAY FROM (CURRENT_DATE - MAX(invoicedate)))
4 FROM ecommerce;
```

The right sidebar shows the database structure for the 'ecommerce' database, listing columns: invoiceno (text), stockcode (text), description (text), quantity (integer), invoicedate (timestamp), unitprice (numeric(12,2)), and customerid (text).

The bottom panel shows the query result (Result 11) with 1 row. The columns are 'latest_transaction_date' and 'days_since_last_record'. The data row shows the date '2011-12-09 12:50:00.000000' and the value '5181'.

5. Accuracy check

If quantity < 0 (return), then invoice number must begin with 'C'.
If quantity > 0 (sale), then invoice number cannot begin with 'C'.
we display rows that violate the rule.

Critical severity level - directly affects financial performance.

```
SELECT *
FROM ecommerce
WHERE
    (Quantity < 0 AND InvoiceNo NOT LIKE 'C%')
    OR (Quantity > 0 AND InvoiceNo LIKE 'C%');
```

Output postgres.public.ecommerce Tx

1-500 of 501+

Tx: Auto

DDL

CSV

	invoiceno	stockcode	description	quantity	invoiceda
1	536589	21777	<null>	-10	2010-12-01
2	536764	849520	<null>	-38	2010-12-02
3	541000	21632	<null>	-41	2011-01-13
4	536996	22712	<null>	-20	2010-12-03
5	536997	22028	<null>	-20	2010-12-03
6	536998	85067	<null>	-6	2010-12-03